T3000 API & Protocol

Data Points API · built Nov 13, 2026

Data Points API

Overview

The Data Points API provides access to inputs, outputs, and variables on BACnet devices. Use these endpoints to read sensor values, control outputs, and manage internal variables.

Point Types

- **Inputs**: Sensor readings (analog: temperature, pressure; binary: switch states)
- **Outputs**: Controllable points (analog: damper position, valve; binary: fan on/off)
- **Variables**: Internal memory locations for calculations and logic

Common Operations

Get All Inputs

TYPESCRIPT

```typescript
const inputs = await fetch('http://localhost:9103/api/t3_device/devices/1/input-points')
  .then(res => res.json());

console.log(inputs);
// [
//   {
//     "id": 1,
//     "device_id": 1,
//     "panel": 5,
//     "point_number": 1,
//     "label": "Room Temperature",
//     "value": 72.5,
//     "auto_manual": 0,
//     "units": "Deg F",
//     "range": 3,
//     "calibration": 0,
//     "filter": 3,
//     "status": "OK"
//   }
// ]
```

Get All Outputs

TYPESCRIPT

```typescript
const outputs = await fetch('http://localhost:9103/api/t3_device/devices/1/output-points
  .then(res => res.json());
```

Get All Variables

TYPESCRIPT

```typescript
const variables = await fetch('http://localhost:9103/api/t3_device/devices/1/variable-pc
  .then(res => res.json());
```

Update Single Output

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/outputs/5000000', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    value: 50.0,
    auto_manual: 1  // Manual mode
  })
});
```

Batch Update

Update multiple points in one request:

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/outputs/batch', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    updates: [
      { objectinstance: 5000000, value: 50.0 },
      { objectinstance: 5000001, value: 75.0 },
      { objectinstance: 5000002, value: 100.0 }
    ]
  })
});
```

Refresh from Device

Force read from hardware:

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/devices/1/inputs/refresh', {
  method: 'POST'
});
```

React Component Usage

Point Display Component

TYPESCRIPT

```typescript
import { usePointStore } from '@/stores/pointStore';

export function InputList({ deviceId }: { deviceId: number }) {
  const inputs = usePointStore(state => state.inputs);
  const loadInputs = usePointStore(state => state.loadInputs);

  useEffect(() => {
    loadInputs(deviceId);
  }, [deviceId, loadInputs]);

  return (
    <table>
      <thead>
        <tr>
          <th>Point</th>
          <th>Label</th>
          <th>Value</th>
          <th>Units</th>
        </tr>
      </thead>
      <tbody>
        {inputs.map(input => (
          <tr key={input.id}>
            <td>{input.point_number}</td>
            <td>{input.label}</td>
            <td>{input.value}</td>
            <td>{input.units}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

Output Control Component

```typescript
export function OutputControl({ output }: { output: Output }) {
  const [value, setValue] = useState(output.value);

  const handleUpdate = async () => {
    await fetch(`http://localhost:9103/api/t3_device/outputs/${output.objectinstance}`,
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ value })
    });
  };

  return (
    <div>
      <label>{output.label}</label>
      <input
        type="number"
        value={value}
        onChange={(e) => setValue(Number(e.target.value))}
      />
      <button onClick={handleUpdate}>Update</button>
    </div>
  );
}
```

Input Point Endpoints

GET /api/t3_device/devices/:id/input-points

Description: Get all inputs for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/input-points HTTP/1.1
```

Response:

```JSON
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "point_number": 1,
    "label": "Room Temperature",
    "value": 72.5,
    "auto_manual": 0,
    "units": "Deg F",
    "range": 3,
    "calibration": 0.0,
    "filter": 3,
    "status": "OK",
    "objectinstance": 3000000,
    "decom": 0,
    "type": "analog"
  }
]
```

Used By:

- InputTable.tsx
- PointGrid.tsx

Handler: `api/src/t3_device/routes.rs::get_input_points`

GET /api/t3_device/inputs/:objectinstance

Description: Get single input by object instance

Parameters:

- `objectinstance` (path, integer) - BACnet object instance (3000000+)

Request:

```HTTP
GET /api/t3_device/inputs/3000001 HTTP/1.1
```

Response:

```JSON
{
  "id": 2,
  "device_id": 1,
  "panel": 5,
  "point_number": 2,
  "label": "Supply Air Temp",
  "value": 55.0,
  "auto_manual": 0,
  "units": "Deg F",
  "range": 3,
  "objectinstance": 3000001
}
```

Used By:

- PointDetail.tsx

Errors:

- `404` - Input not found

PUT /api/t3_device/inputs/:objectinstance

Description: Update input properties (label, calibration, filter, etc.)

Parameters:

- `objectinstance` (path, integer) - BACnet object instance

Request:

```HTTP
PUT /api/t3_device/inputs/3000001 HTTP/1.1
Content-Type: application/json

{
  "label": "Supply Air Temperature",
  "calibration": 0.5,
  "filter": 5
}
```

Updatable Fields:

- `label`, `calibration`, `filter`, `status`, `decom`

Response:

```JSON
{
  "id": 2,
  "label": "Supply Air Temperature",
  "calibration": 0.5,
  "filter": 5
  // ... all fields
}
```

Used By:

- EditInputDialog.tsx

POST /api/t3_device/devices/:id/inputs/refresh

Description: Refresh all inputs from device hardware

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
POST /api/t3_device/devices/1/inputs/refresh HTTP/1.1
```

Response:

```JSON
{
  "success": true,
  "device_id": 1,
  "points_refreshed": 16,
  "duration_ms": 342
}
```

Used By:

- RefreshPointsButton.tsx

Errors:

- `503` - Device offline

Output Point Endpoints

GET /api/t3_device/devices/:id/output-points

Description: Get all outputs for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1/output-points HTTP/1.1
```

Response:

JSON


```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "point_number": 1,
    "label": "Cooling Valve",
    "value": 50.0,
    "auto_manual": 0,
    "units": "Percent",
    "range": 7,
    "objectinstance": 5000000,
    "pwm_period": 15,
    "decom": 0,
    "type": "analog"
  }
]
```


Used By:

- OutputTable.tsx

GET /api/t3_device/outputs/:objectinstance

Description: Get single output by object instance

Parameters:

- `objectinstance` (path, integer) - BACnet object instance (5000000+)

Request:

HTTP

```
GET /api/t3_device/outputs/5000000 HTTP/1.1
```

Response:

JSON

```
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "point_number": 1,
  "label": "Cooling Valve",
  "value": 50.0,
  "auto_manual": 0,
  "objectinstance": 5000000
}
```

Used By:

- OutputControl.tsx

PUT /api/t3_device/outputs/:objectinstance

Description: Update output value and properties

Parameters:

- `objectinstance` (path, integer) - BACnet object instance

Request:

HTTP

```
PUT /api/t3_device/outputs/5000000 HTTP/1.1
Content-Type: application/json

{
  "value": 75.0,
  "auto_manual": 1
}
```

Updatable Fields:

- `value` - Output value (0-100 for analog)
- `auto_manual` - 0=Auto, 1=Manual
- `label`, `pwm_period`, `decom`

Response:

```JSON
{
  "success": true,
  "objectinstance": 5000000,
  "value": 75.0,
  "updated_at": "2026-01-09T10:00:00Z"
}
```

Used By:

- OutputControl.tsx
- ManualOverride.tsx

Errors:

- `400` - Invalid value or mode
- `404` - Output not found
- `503` - Device offline

PUT /api/t3_device/outputs/batch

Description: Batch update multiple outputs

Request:

```HTTP
PUT /api/t3_device/outputs/batch HTTP/1.1
Content-Type: application/json

{
  "updates": [
    { "objectinstance": 5000000, "value": 50.0, "auto_manual": 1 },
    { "objectinstance": 5000001, "value": 75.0, "auto_manual": 1 },
    { "objectinstance": 5000002, "value": 100.0, "auto_manual": 0 }
  ]
}
```

Response:

JSON

```
{
  "success": true,
  "updated": 3,
  "failed": 0,
  "results": [
    { "objectinstance": 5000000, "success": true },
    { "objectinstance": 5000001, "success": true },
    { "objectinstance": 5000002, "success": true }
  ]
}
```

Used By:

- BatchControl.tsx

POST /api/t3_device/devices/:id/outputs/refresh

Description: Refresh all outputs from device hardware

Parameters:

- `id` (path, integer) - Device ID

Request:

HTTP

```
POST /api/t3_device/devices/1/outputs/refresh HTTP/1.1
```

Response:

JSON

```
{
  "success": true,
  "device_id": 1,
  "points_refreshed": 8,
  "duration_ms": 215
}
```

Used By:

- RefreshPointsButton.tsx

Variable Point Endpoints

GET /api/t3_device/devices/:id/variable-points

Description: Get all variables for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1/variable-points HTTP/1.1
```

Response:

```JSON
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "point_number": 1,
    "label": "Setpoint",
    "value": 72.0,
    "auto_manual": 0,
    "units": "Deg F",
    "range": 3,
    "objectinstance": 2000000
  }
]
```

Used By:

- VariableTable.tsx

GET /api/t3_device/variables/:objectinstance

Description: Get single variable by object instance

Parameters:

- `objectinstance` (path, integer) - BACnet object instance (2000000+)

Request:

```HTTP
GET /api/t3_device/variables/2000000 HTTP/1.1
```

Response:

```JSON
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "point_number": 1,
  "label": "Setpoint",
  "value": 72.0,
  "objectinstance": 2000000
}
```

PUT /api/t3_device/variables/:objectinstance

Description: Update variable value and properties

Parameters:

- `objectinstance` (path, integer) - BACnet object instance

Request:

```HTTP
PUT /api/t3_device/variables/2000000 HTTP/1.1
Content-Type: application/json

{
  "value": 70.0,
  "auto_manual": 1
}
```

Response:

```JSON
{
  "success": true,
  "objectinstance": 2000000,
  "value": 70.0
}
```

Used By:

- VariableControl.tsx

PUT /api/t3_device/variables/batch

Description: Batch update multiple variables

Request:

```HTTP
PUT /api/t3_device/variables/batch HTTP/1.1
Content-Type: application/json

{
  "updates": [
    { "objectinstance": 2000000, "value": 70.0 },
    { "objectinstance": 2000001, "value": 65.0 }
  ]
}
```

Response:

```JSON
{
  "success": true,
  "updated": 2,
  "failed": 0
}
```

Used By:

- BatchControl.tsx

POST /api/t3_device/devices/:id/variables/refresh

Description: Refresh all variables from device hardware

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
POST /api/t3_device/devices/1/variables/refresh HTTP/1.1
```

Response:

```JSON
{
  "success": true,
  "device_id": 1,
  "points_refreshed": 64,
  "duration_ms": 892
}
```

Implementation Details

Database Schema

SQL

```sql
CREATE TABLE INPUTS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    point_number INTEGER NOT NULL,
    label TEXT,
    value REAL DEFAULT 0,
    auto_manual INTEGER DEFAULT 0,
    units TEXT,
    range INTEGER,
    calibration REAL DEFAULT 0,
    filter INTEGER DEFAULT 3,
    status TEXT,
    objectinstance INTEGER UNIQUE,
    decom INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE OUTPUTS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    point_number INTEGER NOT NULL,
    label TEXT,
    value REAL DEFAULT 0,
    auto_manual INTEGER DEFAULT 0,
    units TEXT,
    range INTEGER,
    pwm_period INTEGER DEFAULT 15,
    objectinstance INTEGER UNIQUE,
    decom INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE VARIABLES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    point_number INTEGER NOT NULL,
    label TEXT,
```

```
    value REAL DEFAULT 0,
    auto_manual INTEGER DEFAULT 0,
    units TEXT,
    range INTEGER,
    objectinstance INTEGER UNIQUE,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);
```

Object Instance Mapping

```
Inputs:    3000000 + point_number
Outputs:   5000000 + point_number
Variables: 2000000 + point_number
```

Rust Handler Example

RUST

```rust
// Update output with BACnet write
async fn update_output(
    Path(objectinstance): Path<i32>,
    State(state): State<T3AppState>,
    Json(payload): Json<UpdateOutputRequest>,
) -> Result<Json<UpdateResponse>, ApiError> {
    let conn = state.get_conn().await?;

    // Find output
    let output = Output::find()
        .filter(output::Column::Objectinstance.eq(objectinstance))
        .one(&conn)
        .await?
        .ok_or(ApiError::NotFound("Output not found".into()))?;

    // Write to BACnet device via FFI
    let result = unsafe {
        write_bacnet_property(
            output.device_id,
            objectinstance,
            payload.value,
        )
    };

    if result != 0 {
        return Err(ApiError::DeviceOffline);
    }

    // Update database
    let mut active_model: output::ActiveModel = output.into();
    active_model.value = Set(payload.value);
    if let Some(am) = payload.auto_manual {
        active_model.auto_manual = Set(am);
    }

    active_model.update(&conn).await?;

    Ok(Json(UpdateResponse {
        success: true,
        objectinstance,
```

```
        value: payload.value,
        updated_at: Utc::now(),
    }))
}
```

TypeScript Types

TYPESCRIPT

```typescript
export interface Input {
  id: number;
  device_id: number;
  panel: number;
  point_number: number;
  label: string;
  value: number;
  auto_manual: number;
  units: string;
  range: number;
  calibration: number;
  filter: number;
  status: string;
  objectinstance: number;
  decom: number;
}

export interface Output {
  id: number;
  device_id: number;
  panel: number;
  point_number: number;
  label: string;
  value: number;
  auto_manual: number;
  units: string;
  range: number;
  pwm_period: number;
  objectinstance: number;
  decom: number;
}

export interface Variable {
  id: number;
  device_id: number;
  panel: number;
  point_number: number;
  label: string;
  value: number;
  auto_manual: number;
```

```
  units: string;
  range: number;
  objectinstance: number;
}
```

Auto/Manual Modes

- `0` = Auto (device controlled)
- `1` = Manual (user override)

Range Codes

Common range values:

- `3` = Deg F
- `4` = Deg C
- `7` = Percent
- `21` = PPM
- `62` = PSI

See unit conversion tables for complete mapping.

Error Codes

- `POINT_NOT_FOUND` - Object instance doesn't exist
- `DEVICE_OFFLINE` - Cannot communicate with device
- `INVALID_VALUE` - Value out of range
- `READ_ONLY` - Point not writable
- `WRITE_FAILED` - BACnet write error